```python
# SECTION 1: SETUP AND ROAD NETWORK CREATION
!pip install osmnx networkx numpy matplotlib tqdm --quiet
import osmnx as ox
import networkx as nx
import numpy as np
import random
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.lines import Line2D
from tqdm import tqdm
import time

ox.settings.log_console = False
ox.settings.use_cache = True

place_name = "Chennai, Tamil Nadu, India"
G = ox.graph_from_place(place_name, network_type="drive")
G = ox.truncate.largest_component(G)
print("Road network loaded.")
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/104.4 kB ? eta -:--:--
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 104.4/104.4 kB 3.3 MB/s eta
0:00:00
```

```python
# SECTION 2: SHELTER EXTRACTION AND FLOOD SIMULATION

tags = {"amenity": "hospital"}

try:
    shelters = ox.features_from_place(place_name, tags)
except:
    shelters = None

shelter_nodes = []

if shelters is not None and not shelters.empty:
    named = shelters[shelters["name"].notna()] if "name" in
shelters.columns else shelters
    named = named.head(10)
    for idx, row in named.iterrows():
        try:
            point = (row.geometry.centroid.y, row.geometry.centroid.x)
            nearest_node = ox.distance.nearest_nodes(G, point[1],
point[0])
            shelter_nodes.append(nearest_node)
        except:
            continue

shelter_nodes = list(set(shelter_nodes))
```

```python
if len(shelter_nodes) == 0:
    print("No shelters found, using random nodes as shelters.")
    shelter_nodes = random.sample(list(G.nodes), 5)

print("Number of shelters:", len(shelter_nodes))

for u, v, k, data in G.edges(keys=True, data=True):
    data["rainfall"] = random.uniform(0, 1)
    data["elevation"] = random.uniform(0, 1)
    data["drainage"] = random.uniform(0, 1)
    data["soil"] = random.uniform(0, 1)

print("Flood features simulated.")
```

```
Number of shelters: 10
Flood features simulated.
```

```python
# SECTION 3- PSO FOR FLOOD SEVERITY OPTIMIZATION

def fitness(weights):
    error = 0
    for u, v, k, data in G.edges(keys=True, data=True):
        predicted = (
            weights[0] * data["rainfall"]
            + weights[1] * data["elevation"]
            + weights[2] * data["drainage"]
            + weights[3] * data["soil"]
        )
        true_value = 0.5 * data["rainfall"] + 0.3 * data["elevation"]
        error += (predicted - true_value) ** 2
    return error

particles = 20
iterations = 25
dimensions = 4

positions = np.random.rand(particles, dimensions)
velocities = np.random.rand(particles, dimensions)

pbest = positions.copy()
pbest_scores = np.array([fitness(p) for p in positions])
gbest = pbest[np.argmin(pbest_scores)]

w, c1, c2 = 0.5, 1.5, 1.5

for _ in tqdm(range(iterations), desc="Running PSO"):
    for i in range(particles):
        velocities[i] = (
            w * velocities[i]
            + c1 * random.random() * (pbest[i] - positions[i])
            + c2 * random.random() * (gbest - positions[i])
```

```
        )
        positions[i] += velocities[i]
        score = fitness(positions[i])
        if score < pbest_scores[i]:
            pbest[i] = positions[i]
            pbest_scores[i] = score
    gbest = pbest[np.argmin(pbest_scores)]

print("Optimized Flood Weights:", gbest)

for u, v, k, data in G.edges(keys=True, data=True):
    data["flood"] = (
        gbest[0] * data["rainfall"]
        + gbest[1] * data["elevation"]
        + gbest[2] * data["drainage"]
        + gbest[3] * data["soil"]
    )


start_node = random.choice(list(G.nodes))
target_shelter = random.choice(shelter_nodes)

print("Start node:", start_node)
print("Target shelter:", target_shelter)


def get_edge_cost(G, u, v):
    edges = G.get_edge_data(u, v)
    min_cost = float("inf")
    for key in edges:
        edge = edges[key]
        cost = edge["length"] + 10 * edge["flood"]
        if cost < min_cost:
            min_cost = cost
    return min_cost
```

```
Running PSO: 100%|██████████| 25/25 [03:21<00:00,  8.07s/it]

Optimized Flood Weights: [ 0.5059835   0.29325808  0.00722429 -
0.00627357]
Start node: 9911196434
Target shelter: 254141777
```

```
# SECTION 4: GENETIC ALGORITHM ROUTE

def ga_route(G, start, end, population_size=50, generations=40,
mutation_rate=0.2):
    """
    Genetic Algorithm for flood-aware evacuation routing.
    Each chromosome is a path from start to end.
    Fitness = inverse of total cost (length + flood penalty).
```

```
    """
    print("\n[GA] Initializing population via random walks + Dijkstra
seeds...")

    try:
        dijkstra_seed = nx.shortest_path(G, start, end,
weight="length")
    except:
        dijkstra_seed = None

    try:
        flood_aware_seed = nx.shortest_path(
            G, start, end,
            weight=lambda u, v, d: min(
                d2.get("length", 100) + 10 * d2.get("flood", 0)
                for d2 in G.get_edge_data(u, v).values()
            )
        )
    except:
        flood_aware_seed = dijkstra_seed

    def path_cost(path):
        if len(path) < 2:
            return float("inf")
        total = 0
        for u, v in zip(path[:-1], path[1:]):
            if G.has_edge(u, v):
                total += get_edge_cost(G, u, v)
            else:
                total += 1e9
        return total

    def random_walk_path(start, end, max_steps=1500):
        """Random walk that tries to reach end within max_steps."""
        path = [start]
        current = start
        visited = set([start])
        for _ in range(max_steps):
            neighbors = list(G.neighbors(current))
            if not neighbors:
                break
            unvisited = [n for n in neighbors if n not in visited]
            if not unvisited:
                unvisited = neighbors
            current = random.choice(unvisited)
            path.append(current)
            visited.add(current)
            if current == end:
                return path
        if path[-1] != end and dijkstra_seed:
```

```
            try:
                tail = nx.shortest_path(G, path[-1], end,
weight="length")
                path = path + tail[1:]
            except:
                pass
        return path

    def crossover(parent1, parent2):
        """
        Order-based crossover: find common nodes, swap tails.
        """
        common = set(parent1) & set(parent2) - {start, end}
        if not common:
            return parent1[:]
        pivot = random.choice(list(common))
        if pivot in parent1 and pivot in parent2:
            idx1 = parent1.index(pivot)
            idx2 = parent2.index(pivot)
            child = parent1[:idx1 + 1] + parent2[idx2 + 1:]
            return child
        return parent1[:]

    def mutate(path):
        """
        Mutation: replace a sub-segment with a random walk or Dijkstra
path.
        """
        if len(path) < 4:
            return path
        idx = random.randint(1, len(path) - 3)
        node = path[idx]
        try:
            new_tail = nx.shortest_path(G, node, end, weight="length")
            return path[:idx] + new_tail
        except:
            return path

    population = []
    if dijkstra_seed:
        population.append(dijkstra_seed[:])
    if flood_aware_seed:
        population.append(flood_aware_seed[:])

    while len(population) < population_size:
        p = random_walk_path(start, end)
        if p[-1] == end:
            population.append(p)

    while len(population) < population_size:
```

```
        population.append(dijkstra_seed[:] if dijkstra_seed else
[start, end])

    best_path = None
    best_cost = float("inf")

    for gen in tqdm(range(generations), desc="Running GA"):
        costs = [path_cost(p) for p in population]
        fitness_scores = [1.0 / (c + 1e-6) for c in costs]
        total_fitness = sum(fitness_scores)

        min_cost_idx = np.argmin(costs)
        if costs[min_cost_idx] < best_cost:
            best_cost = costs[min_cost_idx]
            best_path = population[min_cost_idx][:]
            print(f"  ✓ GA new best at gen {gen}:
cost={round(best_cost, 2)}, nodes={len(best_path)}")

        probs = [f / total_fitness for f in fitness_scores]
        selected_indices = np.random.choice(
            len(population), size=population_size, replace=True,
p=probs
        )
        selected = [population[i] for i in selected_indices]

        new_population = []

        sorted_pop = sorted(zip(costs, population), key=lambda x:
x[0])
        new_population.append(sorted_pop[0][1][:])
        new_population.append(sorted_pop[1][1][:])

        while len(new_population) < population_size:
            p1, p2 = random.sample(selected, 2)
            child = crossover(p1, p2)
            if random.random() < mutation_rate:
                child = mutate(child)
            if child[-1] != end:
                try:
                    tail = nx.shortest_path(G, child[-1], end,
weight="length")
                    child = child + tail[1:]
                except:
                    child = dijkstra_seed[:] if dijkstra_seed else
child
            new_population.append(child)

        population = new_population

    if best_path is None:
```

```
        print("[GA] Failed — using Dijkstra fallback")
        return dijkstra_seed if dijkstra_seed else [start]

    print(f"[GA] Best path: {len(best_path)} nodes,
cost={round(best_cost, 2)}")
    return best_path

# SECTION 4: ACO ROUTE

def aco_route(G, start, end, ants=30, iterations=30):
    pheromone = {}
    for u, v in G.edges():
        pheromone[(u, v)] = 1.0
        pheromone[(v, u)] = 1.0

    best_path = None
    best_cost = float("inf")

    try:
        fallback = nx.shortest_path(G, start, end, weight="length")
    except:
        fallback = None

    for iteration in range(iterations):
        for ant in range(ants):
            current = start
            visited = set([current])
            path = []
            cost = 0
            max_steps = 2000
            steps = 0

            while current != end and steps < max_steps:
                steps += 1
                neighbors = [n for n in G.neighbors(current) if n not
in visited]
                if not neighbors:
                    neighbors = list(G.neighbors(current))
                    if not neighbors:
                        break

                probabilities = []
                for n in neighbors:
                    tau = pheromone.get((current, n), 1.0)
                    edge_cost = get_edge_cost(G, current, n)
                    eta = 1.0 / (edge_cost + 1e-6)
                    probabilities.append((tau ** 1.0) * (eta ** 2.0))

                probabilities = np.array(probabilities)
                total = probabilities.sum()
```

```
                if total == 0:
                    probabilities = np.ones(len(probabilities)) /
len(probabilities)
                else:
                    probabilities /= total

                if random.random() < 0.2 and fallback and current in
fallback:
                    idx = fallback.index(current)
                    if idx + 1 < len(fallback):
                        next_node = fallback[idx + 1]
                    else:
                        next_node = np.random.choice(neighbors,
p=probabilities)
                else:
                    next_node = np.random.choice(neighbors,
p=probabilities)

                cost += get_edge_cost(G, current, next_node)
                path.append((current, next_node))
                visited.add(next_node)
                current = next_node

            if current == end and cost < best_cost:
                best_cost = cost
                best_path = path
                print(f"  ✓ ACO found path at iteration {iteration},
cost={round(best_cost,2)}, steps={len(best_path)}")

        for edge in pheromone:
            pheromone[edge] *= 0.9

        if best_path:
            for edge in best_path:
                pheromone[edge] = pheromone.get(edge, 1.0) + (1.0 /
best_cost)

    if best_path is None:
        print("ACO failed — using Dijkstra fallback")
        return fallback if fallback else [start]

    route = [start]
    for edge in best_path:
        route.append(edge[1])
    return route

print("\n" + "="*50)
print("RUNNING GENETIC ALGORITHM")
print("="*50)
start_time = time.time()
```

```python
ga_route_path = ga_route(G, start_node, target_shelter)
ga_time = time.time() - start_time

print("\n" + "="*50)
print("RUNNING ANT COLONY OPTIMIZATION")
print("="*50)
start_time = time.time()
aco_route_path = aco_route(G, start_node, target_shelter)
aco_time = time.time() - start_time

print(f"\nGA path length : {len(ga_route_path)} nodes")
print(f"ACO path length: {len(aco_route_path)} nodes")
```

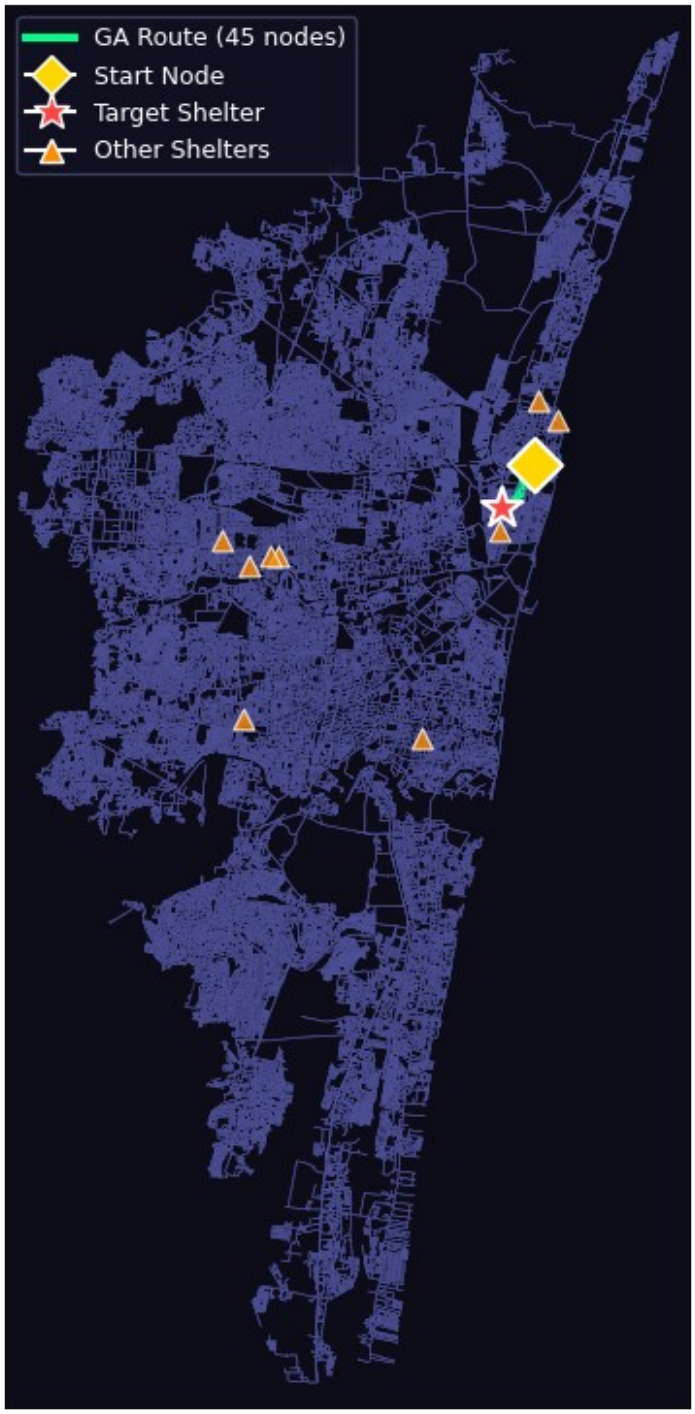
```
==================================================
RUNNING GENETIC ALGORITHM
==================================================

[GA] Initializing population via random walks + Dijkstra seeds...

Running GA:   8%|▉          | 3/40 [00:00<00:03, 12.32it/s]

  ✓ GA new best at gen 0: cost=2516.59, nodes=45

Running GA: 100%|██████████| 40/40 [00:01<00:00, 30.02it/s]

[GA] Best path: 45 nodes, cost=2516.59

==================================================
RUNNING ANT COLONY OPTIMIZATION
==================================================
  ✓ ACO found path at iteration 0, cost=35750.11, steps=934
  ✓ ACO found path at iteration 0, cost=7578.98, steps=145
  ✓ ACO found path at iteration 1, cost=6245.23, steps=117
  ✓ ACO found path at iteration 2, cost=2999.19, steps=58

GA path length : 45 nodes
ACO path length: 59 nodes
```

```python
# SECTION 5: COMPUTE FLOOD SCORES

def compute_flood_score(route):
    if len(route) < 2:
        return float("inf")
    return sum(
        min(G[u][v][k]["flood"] for k in G[u][v])
        for u, v in zip(route[:-1], route[1:])
        if G.has_edge(u, v)
    )

ga_flood = compute_flood_score(ga_route_path)
```

```
aco_flood = compute_flood_score(aco_route_path)

ga_edges = set(zip(ga_route_path[:-1], ga_route_path[1:]))

ec_ga = []
ew_ga = []
for u, v, k in G.edges(keys=True):
    if (u, v) in ga_edges or (v, u) in ga_edges:
        ec_ga.append("#00FF88")
        ew_ga.append(3.5)
    else:
        ec_ga.append("#50529A")
        ew_ga.append(0.4)

fig1, ax1 = ox.plot_graph(
    G,
    edge_color=ec_ga,
    edge_linewidth=ew_ga,
    node_size=0,
    show=False,
    close=False,
    bgcolor="#0d0d1a"
)

ax1.scatter(G.nodes[start_node]['x'], G.nodes[start_node]['y'],
            c='#FFD700', s=200, zorder=6, marker='D',
edgecolors='white', linewidths=1.5)
ax1.scatter(G.nodes[target_shelter]['x'], G.nodes[target_shelter]
['y'],
            c='#FF4444', s=250, zorder=6, marker='*',
edgecolors='white', linewidths=1.5)

for s in shelter_nodes:
    ax1.scatter(G.nodes[s]['x'], G.nodes[s]['y'],
                c='#FF8C00', s=60, zorder=5, alpha=0.8, marker='^',
                edgecolors='white', linewidths=0.8)

legend_elements_ga = [
    Line2D([0], [0], color='#00FF88', linewidth=3, label=f'GA Route
({len(ga_route_path)} nodes)'),
    Line2D([0], [0], marker='D', color='w', markerfacecolor='#FFD700',
           markersize=10, label='Start Node'),
    Line2D([0], [0], marker='*', color='w', markerfacecolor='#FF4444',
           markersize=14, label='Target Shelter'),
    Line2D([0], [0], marker='^', color='w', markerfacecolor='#FF8C00',
           markersize=9, label='Other Shelters'),
]
ax1.legend(handles=legend_elements_ga, loc='upper left',
           facecolor='#111122', labelcolor='white', fontsize=9,
           framealpha=0.9, edgecolor='#444466')
```

```
ax1.set_title(
    f"Chennai Flood Evacuation — GENETIC ALGORITHM\n"
    f"Route Nodes: {len(ga_route_path)}  |  "
    f"Flood Score: {round(ga_flood, 3)}  |  "
    f"Time: {round(ga_time, 2)}s",
    color='white', fontsize=11, pad=12
)
plt.tight_layout()
plt.savefig("chennai_ga_route.png", dpi=150, bbox_inches='tight',
            facecolor='#0d0d1a')
plt.show()
print("Map 1 (GA) saved.")

aco_edges = set(zip(aco_route_path[:-1], aco_route_path[1:])) if
len(aco_route_path) > 1 else set()

ec_aco = []
ew_aco = []
for u, v, k in G.edges(keys=True):
    if (u, v) in aco_edges or (v, u) in aco_edges:
        ec_aco.append("#FF3C78")
        ew_aco.append(3.5)
    else:
        ec_aco.append("#50529A")
        ew_aco.append(0.4)

fig2, ax2 = ox.plot_graph(
    G,
    edge_color=ec_aco,
    edge_linewidth=ew_aco,
    node_size=0,
    show=False,
    close=False,
    bgcolor="#0d0d1a"
)

ax2.scatter(G.nodes[start_node]['x'], G.nodes[start_node]['y'],
            c='#FFD700', s=200, zorder=6, marker='D',
edgecolors='white', linewidths=1.5)
ax2.scatter(G.nodes[target_shelter]['x'], G.nodes[target_shelter]
['y'],
            c='#00CFFF', s=250, zorder=6, marker='*',
edgecolors='white', linewidths=1.5)

for s in shelter_nodes:
    ax2.scatter(G.nodes[s]['x'], G.nodes[s]['y'],
                c='#FF8C00', s=60, zorder=5, alpha=0.8, marker='^',
                edgecolors='white', linewidths=0.8)
```

```
legend_elements_aco = [
    Line2D([0], [0], color='#FF3C78', linewidth=3, label=f'ACO Route
({len(aco_route_path)} nodes)'),
    Line2D([0], [0], marker='D', color='w', markerfacecolor='#FFD700',
           markersize=10, label='Start Node'),
    Line2D([0], [0], marker='*', color='w', markerfacecolor='#00CFFF',
           markersize=14, label='Target Shelter'),
    Line2D([0], [0], marker='^', color='w', markerfacecolor='#FF8C00',
           markersize=9, label='Other Shelters'),
]
ax2.legend(handles=legend_elements_aco, loc='upper left',
           facecolor='#111122', labelcolor='white', fontsize=9,
           framealpha=0.9, edgecolor='#444466')

ax2.set_title(
    f"Chennai Flood Evacuation — ANT COLONY OPTIMIZATION\n"
    f"Route Nodes: {len(aco_route_path)}  |  "
    f"Flood Score: {round(aco_flood, 3)}  |  "
    f"Time: {round(aco_time, 2)}s",
    color='white', fontsize=11, pad=12
)
plt.tight_layout()
plt.savefig("chennai_aco_route.png", dpi=150, bbox_inches='tight',
            facecolor='#0d0d1a')
plt.show()
print("Map 2 (ACO) saved.")

winner = "GA" if ga_flood < aco_flood else "ACO (tie)" if ga_flood ==
aco_flood else "ACO"

print("\n" + "="*50)
print("       PERFORMANCE COMPARISON")
print("="*50)
print(f"{'Metric':<28} {'GA':>10} {'ACO':>10}")
print("-"*50)
print(f"{'Time (seconds)':<28} {round(ga_time,4):>10}
{round(aco_time,4):>10}")
print(f"{'Path Nodes':<28} {len(ga_route_path):>10}
{len(aco_route_path):>10}")
print(f"{'Flood Score':<28} {round(ga_flood,4):>10}
{round(aco_flood,4):>10}")
print("-"*50)
print(f"{'Flood Score Winner':<28} {winner:>22}")
print("="*50)
print(f"\nPSO Optimized Flood Weights: {np.round(gbest, 4)}")
print("  w_rainfall   =", round(gbest[0], 4))
print("  w_elevation  =", round(gbest[1], 4))
print("  w_drainage   =", round(gbest[2], 4))
print("  w_soil       =", round(gbest[3], 4))
print("="*50)
```




```
Map 1 (GA) saved.
```

```
Map 2 (ACO) saved.

=====================================================
       PERFORMANCE COMPARISON
```

```
====================================================
Metric                                   GA        ACO
----------------------------------------------------
Time (seconds)                       3.7301    78.1863
Path Nodes                               45         59
Flood Score                         16.9228    23.3185
----------------------------------------------------
Flood Score Winner                                  GA
====================================================

PSO Optimized Flood Weights: [ 0.506   0.2933  0.0072 -0.0063]
  w_rainfall   = 0.506
  w_elevation  = 0.2933
  w_drainage   = 0.0072
  w_soil       = -0.0063
====================================================
```